Apex POB — Technical Requirements Document (TRD)

Product	Apex POB
Operator / Client	Marconi.ng EPC Limited
Document type	Technical Requirements Document (as-built)
Version	2.0
Audience	Engineering, DevOps, Security, technical stakeholders
Companion docs	`ARCHITECTURE.md`, `docs/INTEGRATIONS.md`, `docs/GOLIVE_HANDOVER.md`, `PRD_ApexPOB.md`

> *As-built technical specification. The authoritative, always-current API contract is the live **OpenAPI/Swagger** spec (`/api/v1/docs` , `openapi.json`). **[Confirm]** marks an operational target the business should ratify.*

---

1. Introduction & scope

This document specifies the architecture, data model, interfaces, integrations and non-functional requirements of Apex POB as delivered. It is the technical baseline for maintenance and future phases.

2. Technology stack

Concern	Technology
Frontend	React (CRA + CRACO), Ant Design, Recharts, Leaflet/react-leaflet, TanStack React Query
Backend	FastAPI (Python), SQLAlchemy ORM, Pydantic
Database	PostgreSQL 15
Cache / broker	Redis 7
Async jobs	Celery (worker + beat)
Proxy / TLS	nginx
Packaging	Docker, docker-compose
Devices	ZKTeco readers (ADMS push protocol)
Observability	Prometheus, Grafana, Alertmanager

3. Architecture

3.1 Logical components

- **nginx** — TLS termination (443) for users; serves the React SPA; reverse-proxies `/api` , `/ws` ; keeps `/iclock/*` on **HTTP 80** for readers; `/monitoring` → Grafana (restricted).

- **frontend** — built React app served by an internal nginx.
- **backend** — FastAPI app (uvicorn, multiple workers) exposing REST + WebSocket; runs background tasks.
- **PostgreSQL** — primary datastore. **Redis** — cache/sessions + Celery broker.
- **celery worker/beat** — scheduled and async jobs.
- **db-backup, prometheus, grafana, alertmanager** — operations.

3.2 Data-flow (reader → system)

```
Reader (ADMS, HTTP:80) → nginx /iclock → backend adms_protocol
   → handle_attlog() routes by reader_purpose:
       ATTENDANCE      → iclock_transaction + zone occupancy + personnel.is_onboard
       ACCESS_ENTRY/EXIT → zone tracking + access_logs + acc_event (+ T&A row)
       MUSTERING       → mustering punch
   → broadcast live updates over WebSocket to dashboards
```

Attendance for payroll is computed (shift/break/reader-purpose aware) into `att_report` and exported by the HR/finance integrations — never raw punches.

3.3 Real-time

WebSocket endpoints for live zone occupancy (`/api/v1/zones/ws`), access events, mustering (`/ws/mustering/...`), emergency (`/api/emergency/ws/...`), notifications (`/ws/notifications`). Client derives `ws/wss` and host from `window.location` ; nginx upgrades via `map $http_upgrade $connection_upgrade` . **TR:** no hardcoded host/port in WS URLs.

4. Data model (key entities)

Entity	Purpose / notes
`personnel`	Employee master; `department_id` (FK) is the authoritative department link; `is_onboard`, `current_location`, `pob_since`. Sync-trigger mirrors to `personnel_employee`.
`personnel_employee`	BioTime-compatible mirror of personnel (kept in sync by DB trigger).
`iclock_transaction`	Raw punch log; `punch_state` (0=in,1=out,2/3=break,4/5=OT,255=auto-detect), `verify_type`, `terminal_sn`.
`att_report`	Computed, shift/break-aware attendance (single source for payroll export).
`iclock_terminal` / `devices`	Reader registry; `zone_id`, `reader_purpose`, `connection_mode` (adms/direct/both), `state`.
`zones`	Zones/locations; `current_personnel_count`, capacity, hazard, lat/long, parent/sub.
`zone_personnel_tracking`	Per-zone CLOCK_IN/OUT events feeding occupancy.
`departments`	Departments; live headcount derived from `personnel.department_id`.
`auth_user`	Authentication users (ZKTeco BioTime format).

`bc_integration_config` / `hr_integration_config`	Encrypted integration config.
`bc_synced_records` / `hr_synced_records`	Idempotency ledger (per employee-day).
`*_sync_log`	Per-run integration audit.

TR-DATA-1: department membership and POB presence are derived from authoritative FKs/latest punch, not legacy association tables. **TR-DATA-2:** implausible punch timestamps (year < 2015 or

> *next year) are rejected at ingest to protect latest-punch logic.*

5. API design

- REST under `/api/v1/*` (and some `/api/*`); JSON; standard verbs; trailing-slash aware.
- **Auth:** `POST /api/v1/auth/login` (form-encoded) → JWT access/refresh; `type=access` allowlisted; optional MFA; refresh + sessions endpoints.
- **Contract:** OpenAPI/Swagger at `/api/v1/docs` (disabled in production), `openapi.json`.
- Representative endpoints: `/api/v1/pob-status/dashboard`, `/api/v1/zones/*`, `/api/v1/departments/*`, `/api/device/terminals/*`, `/api/v1/bc-integration/*`, `/api/v1/hr-integration/*`, `/iclock/cdata|getrequest|devicecmd` (device).
- **TR-API-1:** mutating/admin endpoints enforce RBAC/admin. **TR-API-2:** GET endpoints are side-effect-free.

6. Device integration (ADMS)

- Endpoints at root: `/iclock/cdata` (data+heartbeat), `/iclock/getrequest` (commands), `/iclock/devicecmd` (ack). Served on **HTTP 80** via nginx.
- Auto-registration on first contact (`ZKTECO_AUTO_REGISTER_DEVICES`), comm-key validation, heartbeat/staleness → online/offline.
- `connection_mode` : **`adms`** = push only (remote readers); `direct` / `both` = also TCP-poll port 4370 (same-LAN). **TR-DEV-1:** remote readers must be `adms` to avoid offline flapping. The UI device-edit syncs `connection_mode` to both terminal and device rows.
- ADMS server address is operator-editable (stored in `sys_parameters`), effective without restart.

7. External integrations

7.1 Microsoft Business Central

- OAuth 2.0 client-credentials (Azure AD); cached, lock-serialised token refresh.
- Posts computed daily attendance as `timeRegistrationEntries` to the selected company.
- **TR-INT-BC-1:** payload contains only BC-valid fields (no `idempotencyKey`; BC rejects unknown fields). **TR-INT-BC-2:** idempotency enforced server-side via `bc_synced_records`.

7.2 SeamlessHR

- Config-driven REST (base URL, endpoints, auth header all editable); batched POST.
- **TR-INT-HR-1:** base URL validated (https-only, SSRF guard). **TR-INT-HR-2:** payload shape requires confirmation against the vendor's live API. **[Confirm]**

7.3 Common

- **TR-INT-1:** each (employee, date) exported at most once; not-yet-finalised days skipped; admin `force` / `allow_today` overrides. **TR-INT-2:** credentials encrypted at rest; every run logged.

8. Security requirements

ID	Requirement
TR-SEC-1	All user traffic over TLS (443); HSTS + CSP in production nginx.
TR-SEC-2	JWT access/refresh with strict `type` allowlist; optional MFA; login lockout.
TR-SEC-3	RBAC enforced; admin/global-admin gating on config, integrations, destructive ops.
TR-SEC-4	Integration secrets encrypted at rest (`core/crypto.py`); legacy plaintext tolerated transparently.
TR-SEC-5	Config validators **hard-fail** on default `SECRET_KEY`/`GLOBAL_ADMIN_PASSWORD`/`LICENSE_SECRET` when `ENVIRONMENT=production`.
TR-SEC-6	Outbound integration URLs validated: https-only + reject private/loopback/link-local/metadata (SSRF).
TR-SEC-7	Rate limiting at nginx (api/auth/adms zones) and app layer; client IP from trusted proxy hop.
TR-SEC-8	Audit log of sensitive actions; per-sync audit for HR/finance.
TR-SEC-9	Secrets (`.env.prod`) and TLS keys (`nginx/certs/`) are gitignored; never committed.

9. Non-functional requirements

ID	Category	Requirement / target **[Confirm targets]**
NFR-PERF-1	Performance	Dashboard API responses typically < 500 ms at site scale; live updates within a few seconds of a punch.
NFR-SCAL-1	Scalability	Multiple uvicorn workers; shared HTTP clients; indexed hot tables (`iclock_transaction`, `zone_personnel_tracking`). Hundreds of readers and thousands of personnel supported.
NFR-AVAIL-1	Availability	Containers `restart: unless-stopped`; health checks; nginx fronts the app. Target uptime **[Confirm]**.
NFR-REL-1	Reliability/Safety	Offline readers detected ≤ ~90s; muster-reader-offline safety alert; idempotent payroll export.
NFR-DR-1	Backup/DR	Daily `pg_dump` (7 daily + 4 weekly + 3 monthly); off-box/NAS destination recommended; documented restore.

NFR-OBS-1	Observability	Prometheus metrics, Grafana dashboards (`/monitoring`, LAN/VPN-only), Alertmanager email alerts.
NFR-MAINT-1	Maintainability	Single computed attendance source; shared services; OpenAPI contract; documented env config.
NFR-USAB-1	Usability	Responsive web UI for desktop and field/mobile.
NFR-PORT-1	Portability	Docker-based; per-machine volumes; documented data migration.

10. Infrastructure & deployment

- **Prod:** `docker-compose.prod.yml` + `nginx/conf.d/pob.conf`; services: postgres, redis, db-init, backend, celery-worker, celery-beat, frontend, nginx, db-backup, prometheus, grafana, alertmanager.
- **Networking:** gateway port-forwards public 80/443 → server; `/iclock` on 80 (readers), users on 443. DDNS hostname recommended over bare IP. Server LAN `192.168.0.235`; public `41.67.137.161` (current).
- **TLS:** certs in `nginx/certs/`; Let's Encrypt (DDNS) recommended; self-signed for IP-only (interim).
- **Config:** `.env.prod` (from `.env.prod.example`); `DATABASE_URL` /credentials, `SECRET_KEY`, CORS `BACKEND_CORS_ORIGINS`, `ALLOWED_HOSTS`, `DEVICE_SCAN_SUBNETS`, notification channels.
- **Build note:** frontend build folder is served live in dev; rebuild atomically (temp dir + swap) to avoid serving an empty directory.

11. Background jobs & scheduling

- ZKTeco: heartbeat sweep, subnet auto-discovery, direct-mode live capture / polling (`services/zkteco/*`).
- Nightly HR/finance sync (BC + SeamlessHR) for the finalised previous day.
- Compliance email digest; meeting/MTD scheduled tasks; hourly device time-sync.

12. Constraints & known limitations

- Readers push over **HTTP** (no device TLS); mitigated by operator-controlled link + 443 for users.
- **Verification-method** analytics require readers to report verify mode; not all do → that chart may be empty.
- `255` /auto-detect punches represent presence but not an explicit direction; the attendance engine alternates them by context.
- SeamlessHR wire format is **assumption-based** until validated against the vendor sandbox.
- Feature test suite (beyond security/integration hardening tests) needs restoration. **[Confirm]**

13. Testing strategy

- **Automated:** auth hardening + integration hardening test suites (run `docker exec pob_backend python -m pytest tests/`).
- **Integration:** validate BC and SeamlessHR against vendor sandboxes before go-live (`docs/INTEGRATIONS.md`).
- **Manual/UAT:** exercise each PRD functional requirement; record Pass/Fail (UAT report). **[Confirm]**

14. Glossary

See `PRD_ApexPOB.md` §12. Additionally: **ADMS** push protocol; **att_report** computed attendance; **idempotency ledger** = `*_synced_records`; **reader_purpose** = reader role.